Table of Content

S.No	Topic	Date	Page No.	Sign
1	Implementation of Logic Gates using Python			
2	Fuzzy Logic based Control of Fan Speed according to Room Temperature			
3	Computation of α-Cut and Strong α-Cut of a Fuzzy Set			
4	Determination of Height of a Fuzzy Set and its Classification as Normal / Subnormal			
5	Determination of Support, Core, Boundary and Crossover Points of a Fuzzy Set			
6	Conversion of a Crisp Set into a Fuzzy Set by Assigning Membership Values			
7	Union, Intersection, Complement and Difference of Fuzzy Sets; Fuzzy Relations and Max-Min Composition			
8	Perceptron Training using the Fixed Increment Learning Algorithm			
9	Implementation of Linear Regression			
10	Implementation of Multiple Linear Regression			
11	Implementation of Hebb's Rule for the AND Gate			
12	Implementation of the Delta Rule for the OR Gate			
13	Implementation of the Travelling Salesman Problem using a Genetic Algorithm			
14	Implementation of a Single-Layer Perceptron ANN (without Backpropagation) verified using the AND Gate			

EXPERIMENT 13

Objective:

To write a Python program to solve the Travelling Salesman Problem (TSP) using a Genetic Algorithm.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic knowledge of genetic algorithms and the Travelling Salesman Problem.

Theory:

The Travelling Salesman Problem asks for the shortest possible route that visits every city exactly once and returns to the starting city. It is an NP-hard combinatorial optimisation problem, so for a large number of cities exhaustive search becomes impractical. A Genetic Algorithm (GA) is a population-based search technique inspired by natural selection, and is used to find good (near-optimal) tours efficiently. The main components of the GA used in this program are:

- Chromosome: a route, represented as a permutation of the city indices.
- Fitness: the reciprocal of the total tour length, so that shorter routes have higher fitness.
- Selection: the population is sorted by fitness and parents are chosen from the fitter half; the two best routes are carried over unchanged (elitism).
- Crossover: a segment of one parent is copied into the child and the remaining cities are filled in the order in which they appear in the second parent, so that no city is repeated.
- Mutation: with a small probability, two cities of a route are swapped, which maintains diversity in the population.

PYTHON

```
import random

# Distance matrix between 5 cities
distance = [
    [0, 10, 15, 20, 25],
    [10, 0, 35, 25, 17],
    [15, 35, 0, 30, 20],
    [20, 25, 30, 0, 18],
    [25, 17, 20, 18, 0]
]

cities = list(range(len(distance)))
population_size = 50
generations = 200
mutation_rate = 0.1

def fitness(route):
    total = 0
    for i in range(len(route)):
        total += distance[route[i]][route[(i + 1) % len(route)]]
    return 1 / total

def create_route():
    return random.sample(cities, len(cities))

def crossover(parent1, parent2):
    start, end = sorted(random.sample(range(len(cities)), 2))
    child = [None] * len(cities)
    child[start:end] = parent1[start:end]
    remaining = [city for city in parent2 if city not in child]
    j = 0
    for i in range(len(child)):
        if child[i] is None:
            child[i] = remaining[j]
            j += 1
    return child

def mutate(route):
    if random.random() < mutation_rate:
        i, j = random.sample(range(len(route)), 2)
        route[i], route[j] = route[j], route[i]
    return route
```

```
population = [create_route() for _ in range(population_size)]

for generation in range(generations):
    population.sort(key=fitness, reverse=True)
    new_population = population[:2]  # Keep the best routes
    while len(new_population) < population_size:
        parent1, parent2 = random.choices(
            population[:population_size // 2], k=2)
        child = crossover(parent1, parent2)
        child = mutate(child)
        new_population.append(child)
    population = new_population

best_route = max(population, key=fitness)

total_distance = sum(
    distance[best_route[i]][best_route[(i + 1) % len(best_route)]]
    for i in range(len(best_route))
)

print("\nBest Route:")
print(" -> ".join(str(city + 1) for city in best_route)
      + " -> " + str(best_route[0] + 1))
print("Minimum Distance Found:", total_distance)
```

Output:

```
OUTPUT
Best Route:
5 -> 3 -> 1 -> 2 -> 4 -> 5
Minimum Distance Found: 88
```

Observation:

- The program uses a population of 50 routes, evolved for 200 generations with a mutation rate of 0.1, on a 5-city distance matrix.
- The best route found is $5 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 5$ (returning to city 5) with a total distance of 88.
- The result was cross-checked by exhaustive search of all possible tours, which also gives a minimum distance of 88, so the GA has found the optimal tour.
- Since the algorithm uses random initialisation, crossover and mutation, the route may appear in a different (equivalent) order or direction on different runs, but its length remains the same.

Result:

The Python program successfully solved the Travelling Salesman Problem for the given set of cities using a Genetic Algorithm and obtained the minimum tour distance of 88.

EXPERIMENT 14

Objective:

To implement an Artificial Neural Network (ANN) using a single-layer perceptron without backpropagation and to verify its operation using the AND logic gate.

Prerequisites:

- Set up Python interpreter (PyCharm).
- Basic knowledge of artificial neural networks, activation functions and the perceptron learning rule.

Theory:

A single-layer perceptron consists of an input layer connected directly to one output neuron through adjustable weights. The neuron computes the net input as the weighted sum of its inputs plus a bias and passes it through a step (hard-limiting) activation function:

$$net = w1 \cdot x1 + w2 \cdot x2 + b \text{ , } y = 1 \text{ if } net \geq 0, \text{ otherwise } y = 0$$

Since there is no hidden layer, no backpropagation is required. The weights and bias are corrected directly after every training sample using the perceptron learning rule:

$$w = w + \eta\,(t - y)\,x \text{ , } b = b + \eta\,(t - y)$$

where t is the target output, y is the actual output and η is the learning rate. Training continues for a fixed number of epochs or until an entire epoch produces no errors. As the AND function is linearly separable, the perceptron is guaranteed to converge.

PYTHON

```
inputs = [[0,0],[0,1],[1,0],[1,1]]
targets = [0,0,0,1]

w1 = 0.0
w2 = 0.0
bias = 0.0
learning_rate = 0.1
epochs = 100

def activation(net):
    return 1 if net >= 0 else 0

for epoch in range(epochs):
    errors = 0
    for x, target in zip(inputs, targets):
        net = w1*x[0] + w2*x[1] + bias
        output = activation(net)
        error = target - output
        w1 += learning_rate*error*x[0]
        w2 += learning_rate*error*x[1]
        bias += learning_rate*error
        if error != 0:
            errors += 1
    if errors == 0:
        print("Converged at epoch", epoch + 1)
        break

print("\nAND Gate Results")
print("X1 X2 Target Output")
for x, target in zip(inputs, targets):
    net = w1*x[0] + w2*x[1] + bias
    output = activation(net)
    print(x[0], x[1], " ", target, "    ", output)

print("\nFinal weights:", w1, w2)
print("Final bias:", bias)
```

Output:

```
OUTPUT
Converged at epoch 4

AND Gate Results
X1 X2 Target Output
0 0   0      0
0 1   0      0
1 0   0      0
1 1   1      1

Final weights: 0.2 0.1
Final bias: -0.20000000000000004
```

Observation:

- The weights and bias are initialised to zero, with a learning rate of 0.1 and a maximum of 100 epochs.
- The perceptron made no errors in the fourth epoch, so training stopped (converged) at epoch 4, well before the epoch limit.
- The final weights w1 = 0.2, w2 = 0.1 and bias ≈ −0.2 correctly reproduce the AND truth table: only the input (1, 1) gives output 1 (the bias value −0.20000000000000004 is a floating-point representation of −0.2).

Result:

The single-layer perceptron was successfully trained without backpropagation and correctly implemented the AND gate, as verified by the truth table obtained.